Pontificia Universidad
JAVERIANA
Cali

DIPLOMADO

Arquitectura de Software y Cloud Computing

Educación Continua

Arquitectura del sistema

Diseñando Arquitecturas Resilientes

Documento de arquitectura y resultados · Reto 3: Diseñando Arquitecturas Resilientes · Módulo 3

Grupo 2
Daniel Mazo
Freddy Aparicio
Camilo Céspedes

Noviembre 2026

Contenido

Contenido 2
1. Resumen ejecutivo 3
2. Atributos de calidad y priorización 3
2.1 Restricciones 4
2.2 Matriz de riesgos 4
3. Tácticas de arquitectura 5
4. Decisiones de arquitectura 6
5. Respuestas por nivel (contrato HTTP) 7
6. ¿Qué hace el sistema? 7
7. Opciones consideradas 8
7.1 Opciones de arquitectura 8
7.2 Conclusión 9
8. El plano de control 9
9. Modelo C4 9
9.1 Vista general 9
9.2 Contenedores (detalle de AWS) 10
10. Despliegue en AWS (solución oficial) 12
10.1 Qué se desplegó y cómo 12
10.2 Calentamiento y arranque en frío 13
11. Resultados de la medición 13
11.1 Cómo se midió 13
11.2 Resultados de la corrida oficial (02/11/2026, AWS) 13
12. Disponibilidad: qué pasa si se cae un componente 16
13. Conclusión 16
Anexo A. Glosario 17

1. Resumen ejecutivo

El Reto 3 pide un sistema que se mantenga operativo ante picos de errores: cuando detecta fallas recurrentes debe degradar sus capacidades en tres niveles de servicio (Full, Degradado y Mínimo) y, cuando la salud vuelve a niveles aceptables, recuperarlas de forma automática. Cada cambio de nivel debe quedar registrado en logs o métricas, y todo se prueba con un script de k6 que entrega el profesor y que no se puede modificar.

La solución está desplegada en AWS (us-east-2). API Gateway recibe las peticiones y las pasa a una función router, que numera cada petición, decide su nivel de servicio por "minuto" del k6 (bloques de 20 peticiones) y la envía a una de tres funciones Lambda, una por nivel. El estado compartido (número de petición, nivel, errores del bloque y racha de éxitos) vive en un ítem de DynamoDB, y los cambios de nivel quedan en CloudWatch Logs y en un dashboard.

Todo el documento se organiza alrededor de una decisión: priorizamos la disponibilidad, entendida como resiliencia (seguir respondiendo de forma controlada y recuperarse sin intervención), a costa de un salto adicional por petición y de depender de DynamoDB. Esa decisión, explicada en la sección 2, determina las tácticas (sección 3) y las decisiones de diseño (sección 4). En la corrida oficial del 02/11/2026, el sistema siguió la secuencia 1-2-3-2-1 y terminó en el Nivel 1 (sección 11).

2. Atributos de calidad y priorización

Priorizamos que el sistema siga respondiendo, con un servicio reducido, pero garantizado, cuando los errores aumentan, y que vuelva solo a la normalidad. Es lo que evita que un pico de errores se convierta en una caída total.

Atributo	Qué significa aquí	Evidencia
Disponibilidad / resiliencia (priorizado)	Seguir respondiendo ante errores recurrentes: degradar a un nivel con menos capacidades y, al normalizarse, recuperar. El cliente nunca se queda sin respuesta.	Corrida oficial del k6 (02/11/2026): 140 de 140 peticiones respondidas, 35 con el error que pide el script y ninguna falla inesperada. Secuencia 1-2-3-2-1, con cambios de nivel en las peticiones 21, 61, 126 y 136, y terminó en el Nivel 1.
Observabilidad	Cada cambio de nivel y las métricas clave deben ser visibles en logs o métricas.	Evento CAMBIO_NIVEL en CloudWatch Logs (número de petición, minuto del k6, nivel de origen y de destino, motivo y errores del bloque anterior) y dashboard reto3-resiliencia.
Desempeño (no priorizado)	La carga del reto es baja (unas 20 peticiones por minuto): se mide, pero no se optimiza.	Latencia promedio de 167 ms y p95 de 182 ms vistos por k6 en la corrida oficial (máxima de 356 ms). La primera petición tras un periodo sin tráfico puede tardar más por el arranque en frío (sección 10.2).

El desempeño no se priorizó porque el reto mide la máquina de estados ante errores y no el rendimiento.

Lo que cuesta priorizar así: el router suma un salto a cada petición (≈ 53 ms de promedio medidos en la corrida oficial) y el sistema depende de DynamoDB; el nivel se decide solo con respuestas de error, no con latencia ni con fallas de dependencias; y las rachas de 30 y 40 peticiones sin error para recuperar cada nivel son decisiones nuestras, calibradas con el script oficial, no del enunciado.

2.1 Restricciones

Lo que el reto no deja negociar, y cómo lo cumple el diseño:

Restricción	Cómo la cumplimos
Solo servicios de AWS	API Gateway, Lambda, DynamoDB y CloudWatch, con IAM para los permisos.
El script de k6 no se modifica	Solo cambia la URL del API. El diff de una línea queda guardado como evidencia en cada corrida.
API Gateway como entrada única	Toda petición entra por /service-api y el router la lleva al servicio del nivel vigente.
Mensajes exactos por nivel	Cada servicio responde el texto que pide el enunciado, incluidos los del Nivel 3.
Sin plantilla inicial	El diseño y el código se hicieron desde cero.
Una cuenta de AWS para el equipo	Una persona despliega y el panel entra con un usuario de IAM de solo lectura; nadie más modifica la infraestructura.

2.2 Matriz de riesgos

Cada riesgo se califica por su impacto (1 a 3) y su probabilidad (1 a 3); el resultado es el producto de ambos. De 1 a 2 es bajo, de 3 a 4 medio y de 6 a 9 alto.

Son los riesgos que el diseño atiende, con su calificación antes y después de mitigar y lo que mostró la corrida oficial de la sección 11. R1, R2 y R5 se comprobaron en la corrida; R3 y R4 no se provocaron y quedan descritos en la sección 12; R6 solo podría aparecer con más de un usuario virtual.

Riesgo	Antes	Qué lo baja	Evidencia en la corrida oficial	Después
R1. El sistema oscila entre niveles	9 (alto)	Recuperación con rachas de éxitos: 30 peticiones seguidas sin error para pasar del Nivel 3 al 2 y 40 para pasar del 2 al 1, siempre de a un nivel. Un bloque limpio ya no devuelve el sistema al Nivel 1.	Secuencia 1-2-3-2-1 sin saltos ni oscilaciones (Figura 4).	3 (medio)
R2. Dos instancias responden niveles distintos	6 (alto)	El nivel, el número de petición y la racha se guardan en un solo ítem de DynamoDB, con escritura condicional por versión; todas las instancias lo leen de ahí.	Las 140 peticiones se numeraron de 1 a 140, sin repetidos ni faltantes, y cada “minuto” del k6 respondió un solo nivel salvo el 7, que recorre la recuperación (Figura 4).	3 (medio)

Riesgo	Antes	Qué lo baja	Evidencia en la corrida oficial	Después
R3. El servicio de un nivel falla o se cuelga	6 (alto)	Se corta a los 3 s y sin reintentos: responde el mensaje del Nivel 3, y el error cuenta para el bloque y rompe la racha.	No se provocó en la corrida; el caso está descrito en la sección 12.	2 (bajo)
R4. DynamoDB no responde	3 (medio)	Falla segura: responde “Nivel 3: Sistema bajo mantenimiento” (503), nunca un error crudo.	No se provocó en la corrida; el caso está descrito en la sección 12.	2 (bajo)
R5. Los umbrales 30 y 40 dependen del script	6 (alto)	Se calibraron con el script oficial, con 5 peticiones de margen (la racha va en 35 cuando hacen falta 40, y en 25 cuando hacen falta 30), y son variables de entorno (RACHA_N2 y RACHA_N1) que se cambian sin tocar el código.	Con el script oficial la racha llegó a 30 en la petición 126 y a 40 en la 136, y la corrida terminó en el Nivel 1.	4 (medio)
R6. El ítem único de estado es un punto caliente	4 (medio)	Escritura condicional por versión, con hasta 5 reintentos: si dos escrituras chocan, la segunda relee el estado y recalcula.	No se provocó: con un solo usuario virtual no hay concurrencia. Las 140 peticiones se numeraron de 1 a 140; con más tráfico habría que repartir el estado.	2 (bajo)

3. Tácticas de arquitectura

Cada táctica aplicada en un componente concreto de la solución de AWS:

Táctica	Dónde (AWS)	Cómo ayuda	Atributo
Monitoreo de condición (contar errores por bloque de 20 peticiones)	Router y tabla DynamoDB reto3-estado: un solo ítem con los errores del bloque en curso y la racha de éxitos.	Detecta errores recurrentes sin una pieza externa y decide el nivel del bloque siguiente.	Disponibilidad
Detección de excepciones	Router: invocación de la Lambda del nivel con corte a los 3 s y un solo intento.	Una falla del servicio o una espera larga cuentan como error de salud: no se esconden con reintentos.	Disponibilidad
Degradación	Lambdas *reto3-nivel1*, *reto3-nivel2* y *reto3-nivel3*: todas las capacidades, solo validar y registrar.	Mantiene lo esencial cuando suben los errores.	Disponibilidad
Reconfiguración	Router: envía cada petición a la Lambda del nivel vigente.	Cambia lo que hace el sistema sin tocar al cliente ni el código de los niveles.	Disponibilidad
Manejo de excepciones con falla segura	Router: si DynamoDB no responde, contesta como Nivel 3; si falla la Lambda del nivel, 503 y la cuenta como error.	El cliente siempre recibe una respuesta clara.	Disponibilidad

Táctica	Dónde (AWS)	Cómo ayuda	Atributo
Histéresis (racha de éxitos)	Regla del router: para subir de nivel se exige una racha de peticiones sin error (30 para pasar del Nivel 3 al 2 y 40 para pasar del 2 al 1; variables RACHA_N2 y RACHA_N1).	Un bloque bueno no basta para recuperar: evita que el sistema oscile entre niveles.	Disponibilidad
Reintroducción gradual	Regla del router: se sube de a un nivel (N3 → N2 → N1), nunca de golpe al Nivel 1.	Evita volver de golpe al Nivel 1 con un servicio que apenas se recupera.	Disponibilidad
Externalizar el estado	DynamoDB con escritura condicional por versión del ítem.	Todas las instancias de Lambda ven el mismo nivel y se evita la oscilación entre niveles.	Disponibilidad
Aislar por nivel	Una Lambda por nivel.	Una falla en un nivel no arrastra a los demás.	Disponibilidad
Registrar eventos y métricas	CloudWatch Logs (CAMBIO_NIVEL), métricas EMF (Embedded Metric Format) y dashboard.	Cada transición queda visible y comprobable.	Observabilidad

Las tácticas se pueden leer como tres preguntas: ¿algo está fallando? (monitoreo y detección de excepciones), ¿cómo sigo respondiendo? (degradación y reconfiguración) y ¿cuándo vuelvo a la normalidad? (histéresis y reintroducción gradual).

Juntas se parecen a un circuit breaker de tres estados: el Nivel 1 equivale a cerrado, el Nivel 2 a semiabierto y el Nivel 3 a abierto, con una ventana de un bloque de 20 peticiones. La analogía es parcial: el Nivel 3 no corta el servicio, sigue respondiendo con un mensaje de mantenimiento.

4. Decisiones de arquitectura

Decisión	Atributo	Por qué	A costa de
Un router delante de tres funciones de nivel	Disponibilidad	Un solo punto decide el nivel y cada nivel es un servicio independiente.	Un salto más por petición (≈ 53 ms) y dos invocaciones de Lambda.
Estado en un solo ítem de DynamoDB	Disponibilidad	Las Lambdas no comparten memoria: con varias instancias, un contador local oscilaría entre niveles.	Dos lecturas y dos escrituras en DynamoDB por petición; un solo ítem con versión es un punto caliente: con mucha concurrencia habría choques y reintentos.
El nivel lo deciden los errores del “minuto” anterior (un bloque de 20 peticiones, no el reloj)	Disponibilidad	Es la regla de evaluación del reto: los errores del minuto que pasó deciden el nivel del siguiente. El enunciado no fija la ventana; la definimos nosotros como un bloque de 20 peticiones.	Un bloque se mide en peticiones y no en tiempo: con poco tráfico, 20 peticiones pueden tardar horas. Si el script cambia sus errores, hay que revisar los umbrales.

Decisión	Atributo	Por qué	A costa de
Recuperar con rachas de 30 y 40 peticiones sin error	Disponibilidad	Evita recuperar el Nivel 1 con un servicio que acaba de fallar y que el nivel oscile entre bloques.	Los umbrales 30 y 40 se calibraron con el script oficial, con 5 peticiones de margen: si el script cambia, hay que revisarlos.
Falla segura hacia el Nivel 3	Disponibilidad	Sin estado no se conoce la salud; responder al mínimo es más seguro que adivinar.	Una falla de DynamoDB degrada el servicio, aunque las funciones estén sanas.
Logs JSON con métricas EMF (Embedded Metric Format)	Observabilidad	Cada cambio de nivel queda como evento.	Más volumen de logs por petición.

5. Respuestas por nivel (contrato HTTP)

Nivel	Sin error	Con error ("error": true)	Capacidades
1. Full	200 Nivel 1: OK	500 Nivel 1: Operación Full con Error	Validar, registrar, analizar riesgo y notificar.
2. Degradado	200 Nivel 2: OK	500 Nivel 2: Operación Limitada	Solo validar y registrar.
3. Mínimo	200 Nivel 3: Operación al mínimo	503 Nivel 3: Sistema bajo mantenimiento, intente más tarde	Ninguna.

6. ¿Qué hace el sistema?

Cada petición pasa por el router, que la numera y averigua en qué nivel opera. Ese nivel se decide con los errores del “minuto” anterior, un bloque de 20 peticiones numeradas (no el reloj): con 5 o más errores el sistema opera en el Nivel 2 y con 10 o más en el Nivel 3. La degradación es inmediata (con 10 errores se pasa directo al Nivel 3) y se evalúa al abrir cada bloque. Para recuperar se exige una racha de peticiones seguidas sin error: 30 para pasar del Nivel 3 al 2 y 40 para pasar del 2 al 1, siempre de a un nivel.

Errores del bloque anterior	Nivel del bloque actual
Menos de 5	El nivel vigente; sube un nivel solo si la racha sin error llegó a 30 (desde el Nivel 3) o a 40 (desde el Nivel 2).
5 o más	Nivel 2 si venía en el Nivel 1; si ya estaba en el Nivel 2 o en el 3, se mantiene.
10 o más	Nivel 3, de inmediato.

Con el patrón del script oficial, el bloque 1 (peticiones 1 a 20) manda 5 errores y el bloque siguiente opera en el Nivel 2 (desde la petición 21). El bloque 3 manda 15 errores y desde la petición 61 el sistema opera en el Nivel 3, que se sostiene aunque el bloque 5 traiga otros 15 errores. Con las peticiones 96 a 125 sin error la racha llega a 30 y sube al Nivel 2 (petición 126); con 10 más llega a 40 y sube al Nivel 1 (petición 136): la secuencia es 1-2-3-2-1.

7. Opciones consideradas

Esta sección compara la arquitectura adoptada con otra opción que evaluamos. El reto mide la máquina de estados ante errores y no el rendimiento, por eso la comparación se centra en la previsibilidad de las transiciones, el aislamiento de fallas y la complejidad.

7.1 Opciones de arquitectura

- **Router con estado en DynamoDB (adoptada).** API Gateway entrega la petición a una función router. El router numera la petición y decide su nivel con el estado guardado en DynamoDB, invoca la función del nivel y cuenta el resultado.
- **Una sola función con tres caminos de código**. El mismo router, pero sin funciones separadas por nivel.

Criterio	Router + DynamoDB (adoptada)	Una sola función
Quién enruta	La función router. API Gateway solo entrega la petición.	La propia función, que también atiende cada nivel.
Cómo se decide el nivel	Con los errores del bloque anterior y la racha de éxitos, leídos de DynamoDB en cada petición.	Con la misma regla y el mismo estado en DynamoDB, dentro de una sola función.
Previsibilidad de las transiciones	Alta: el nivel depende solo de los errores del bloque anterior y de la racha, no de la latencia, y se reproduce en simulación.	Igual de alta: la regla es la misma.
Componentes de AWS	API Gateway, 4 Lambda, DynamoDB, CloudWatch.	API Gateway, 1 Lambda, DynamoDB, CloudWatch.
En el camino de cada petición	Router, dos lecturas y dos escrituras en DynamoDB y la función del nivel (≈ 53 ms de promedio medidos en el router en la corrida oficial).	Las mismas llamadas a DynamoDB, sin la invocación a otra función.
Aislamiento de fallas	Una falla en la función de un nivel no afecta a las otras dos.	Una falla o un error de despliegue en la función afecta a los tres niveles a la vez.
Cumplimiento del enunciado	Cumple: un servicio por nivel detrás de API Gateway.	No cumple “un servicio por nivel”: los tres viven en una sola función.
Falla segura	Si DynamoDB no responde, contesta como Nivel 3.	Igual.
Límite de escala	Estado en un solo ítem con versión: del orden de 1.000 escrituras por segundo sobre un mismo ítem y, con concurrencia, choques de versión y reintentos. Con más tráfico habría que repartirlo.	El mismo límite, porque el contador es el mismo.

7.2 Conclusión

- **Se mantiene el router con DynamoDB.** Cumple el objetivo de un servicio por nivel, aísla las fallas de cada nivel y su comportamiento se verificó en AWS.
- **La función tiene un salto menos,** pero junta los tres niveles en un solo despliegue: una falla en esa función deja sin servicio a los tres niveles, y no cumple el requisito de un servicio por nivel.

8. El plano de control

Un panel web local lanza el k6 oficial, permite enviar peticiones a mano, muestra el avance de la corrida y exporta la evidencia. Un interruptor (Local / AWS) cambia la fuente de datos sin tocar el resto del panel: en Local lee LocalStack y en AWS lee DynamoDB y CloudWatch Logs con un usuario de IAM de solo lectura, sin acceso a la consola y limitado a los recursos reto3-*.

Función del panel	Qué muestra o hace	Fuente
Nivel actual	Nivel vigente, errores del bloque en curso, racha hacia el umbral de recuperación y peticiones que faltan para cerrar el bloque.	DynamoDB
Correr k6 oficial	Copia el script con la URL cambiada, verifica con diff que sea la única línea distinta y guarda la salida cruda.	k6
Línea de tiempo por “minuto” del k6	El nivel que vio el k6 frente al que registró el servidor y al que predice la regla, petición por petición, con marca de coherencia por “minuto” del k6.	k6 y CloudWatch Logs
Cambios de nivel	Eventos CAMBIO_NIVEL y fallas del router.	CloudWatch Logs
Prueba manual	Peticiones con o sin error por la misma puerta del k6; permite mostrar el mensaje de mantenimiento del Nivel 3.	API Gateway
Descargas	ZIP con el log del k6, los logs de CloudWatch, los minutos (CSV), el resumen (JSON) y el diff del script.	Todas

En AWS el panel es de solo lectura. El panel observa y no mide: la evidencia sigue siendo el log del k6 y los eventos PETICION y CAMBIO_NIVEL de CloudWatch Logs.

9. Modelo C4

9.1 Vista general

Antes del detalle, así se ve el sistema completo. El script de k6 envía las peticiones a la aplicación en AWS y un panel de estado, de solo lectura, consulta DynamoDB y CloudWatch para mostrar lo que ocurre y exportar la evidencia de la ejecución.

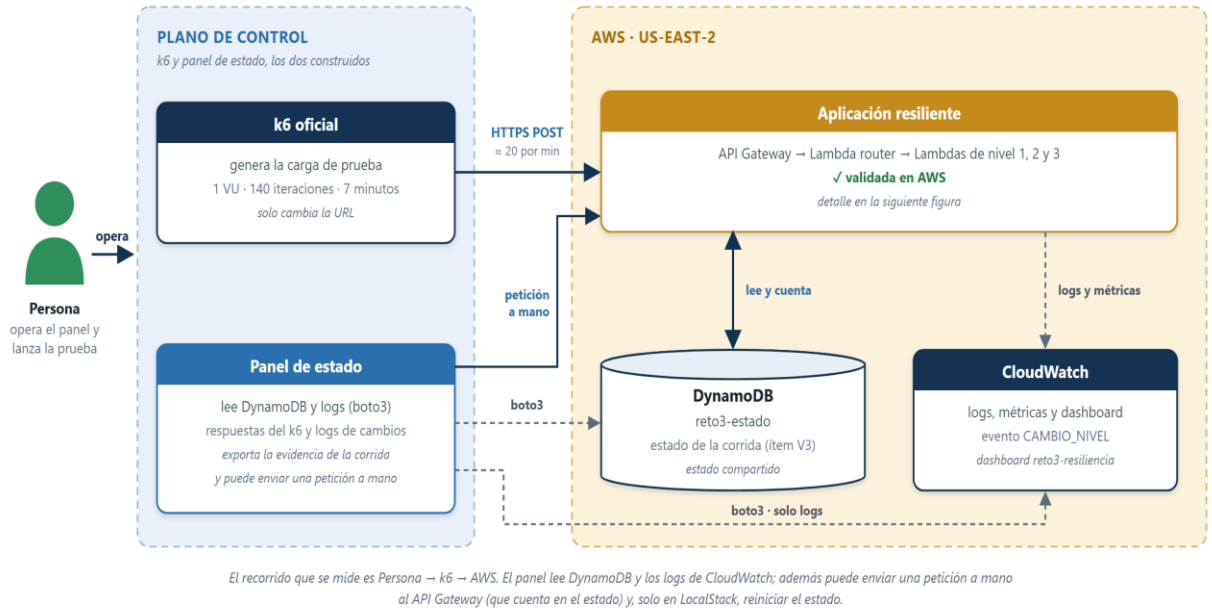


Figura 1. Vista general del Reto 3: el panel de control (k6 y panel de estado) frente a AWS.

9.2 Contenedores (detalle de AWS)

La propuesta recibe las peticiones por un API Gateway, que las pasa a una función router. El router es el monitor del sistema: decide en qué nivel de servicio opera cada petición, según los "minutos" del k6 (bloques de 20 peticiones), y la envía al servicio de ese nivel.

El nivel se calcula con los errores del "minuto" anterior, un bloque de 20 peticiones numeradas: con 5 o más errores el sistema pasa al Nivel 2 (degradado) y con 10 o más al Nivel 3 (mínimo). La degradación es inmediata; para recuperar se exige una racha de peticiones sin error (30 para pasar del Nivel 3 al 2 y 40 para pasar del 2 al 1), siempre de a un nivel. Como las funciones Lambda no comparten memoria entre instancias, el estado de la corrida (número de petición, nivel, errores del bloque y racha) se guarda en un ítem de DynamoDB. Cada cambio de nivel queda registrado en los logs de CloudWatch.

El recorrido de una petición, de izquierda a derecha en la Figura 2 (y paso a paso en la Figura 3):

1. El k6 envía la petición por HTTPS.
2. API Gateway la pasa al router.
3. El router numera la petición, lee el estado en DynamoDB y decide el nivel con los errores del bloque anterior y la racha de éxitos.
4. Invoca solo el servicio del nivel vigente y responde su mensaje.
5. Cada cambio de nivel queda registrado con su motivo en CloudWatch.

El router cumple el papel del patrón Gateway Routing: API Gateway entrega la petición y el router, detrás, decide el nivel. Cuesta unos 53 ms por petición.

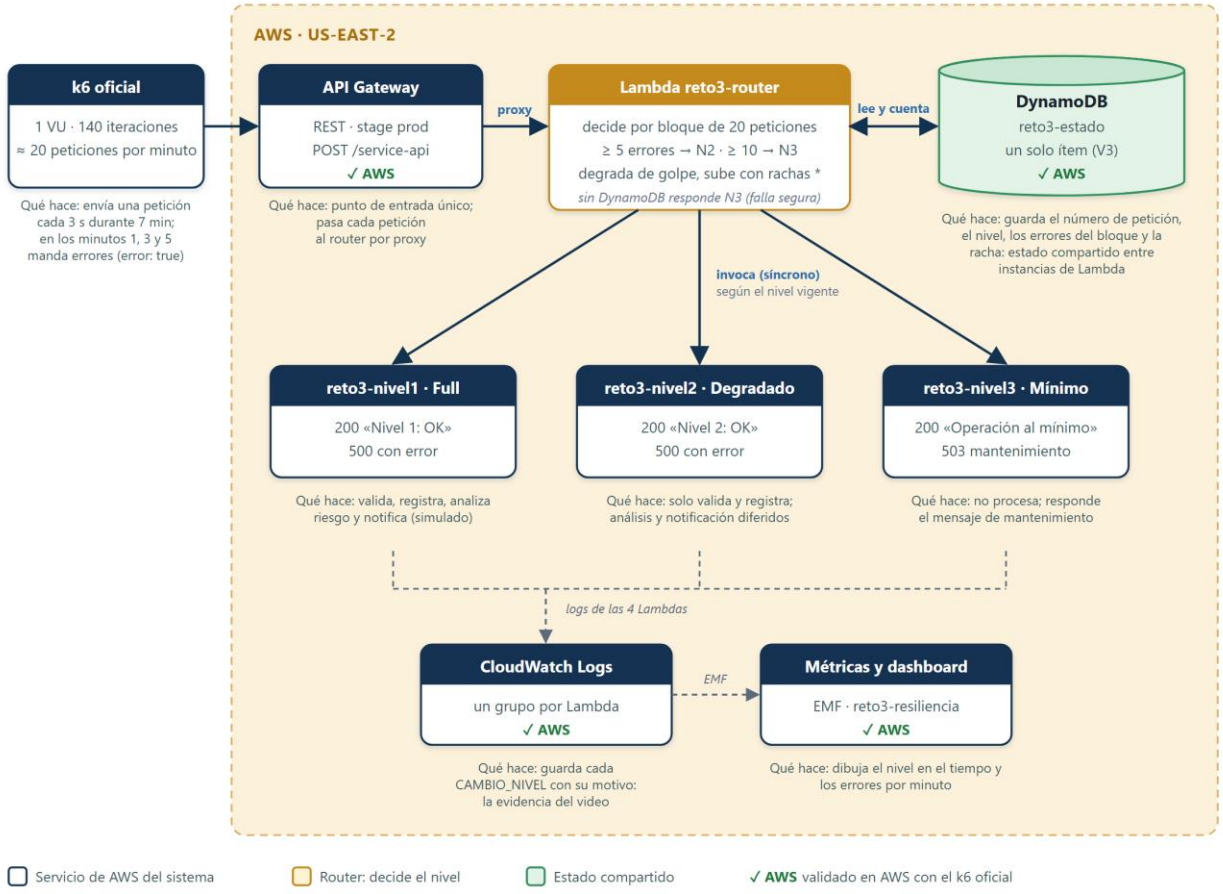


Figura 2. Detalle de AWS: API Gateway, router, servicios de los tres niveles, estado en DynamoDB y observabilidad en CloudWatch. Las piezas con la marca "✓AWS" están validadas con el k6 oficial en AWS.

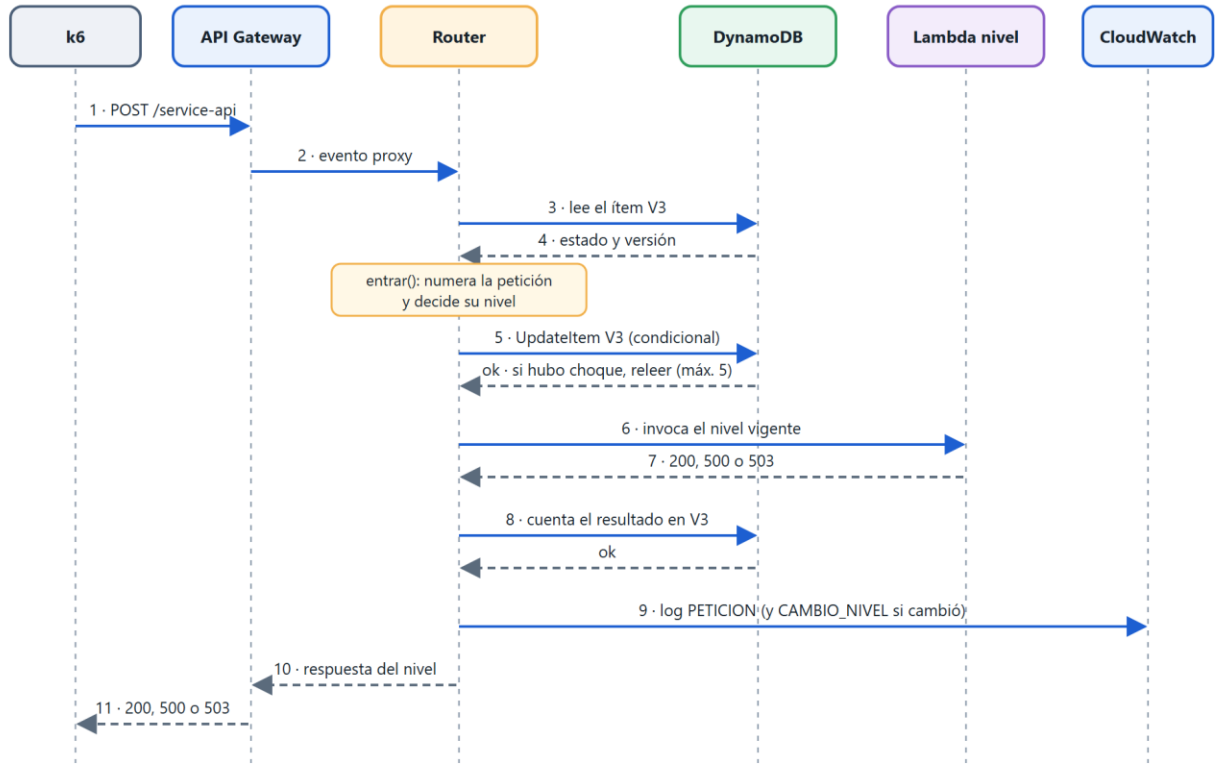


Figura 3. Secuencia de una petición: el router lee y numera con el ítem V3 de DynamoDB, invoca la Lambda del nivel vigente, cuenta el resultado y deja el log.

10. Despliegue en AWS (solución oficial)

10.1 Qué se desplegó y cómo

La solución está en la región us-east-2. Se desplegó a mano por la consola de AWS. La memoria y el tiempo máximo de las Lambdas no vienen del enunciado: son decisiones nuestras.

Recurso	Configuración
API Gateway	REST reto3-api, regional, stage prod. POST /service-api con integración Lambda proxy hacia el router. Límite del stage: 25 r/s y ráfaga de 50 (verificado en el stage prod).
Lambda reto3-router	Python 3.12, arm64, 256 MB, tiempo máximo de 10 s. Variables: TABLA, NIVEL1_FN, NIVEL2_FN, NIVEL3_FN, UMBRAL_DEGRADADO (5), UMBRAL_MINIMO (10), MODO (histeresis), RACHA_N2 (30) y RACHA_N1 (40).
Lambdas reto3-nivel1, 2 y 3	Misma configuración. handler_nivel1, handler_nivel2 y handler_nivel3 (mismo código en las cuatro funciones).
DynamoDB reto3-estado	Clave de partición pk (texto), capacidad bajo demanda, TTL de 24 h. El estado de la corrida vive en un solo ítem, con pk = V3.
IAM	Rol reto3-niveles (logs) y rol reto3-router (logs, leer y actualizar la tabla, invocar las tres funciones de nivel).

Recurso	Configuración
CloudWatch	Cuatro grupos de logs /aws/lambda/reto3-* con retención de 14 días y métricas EMF en el espacio Reto3.
Acceso de lectura	Usuario IAM reto3-panel-lectura, sin consola, con una política de solo lectura limitada a los recursos reto3-*.

10.2 Calentamiento y arranque en frío

El sistema responde con tiempos bajos cuando está en uso: en caliente, el router tarda unos 53 ms en promedio por petición (mediana de 48 ms y percentil 95 de 68 ms en la corrida oficial) y cada función de nivel unos 2 ms. La primera petición después de un tiempo sin tráfico paga el arranque en frío (en una prueba anterior, el router tardó 1.386 ms de ejecución más 257 ms de arranque, y cada función de nivel unos 80 ms de arranque). Por eso, antes de una carga se calientan las cuatro Lambdas: una prueba de cada nivel desde la consola y tres o cuatro peticiones sin error al API; después se esperan más de 15 s antes de lanzar el k6, para que la corrida empiece de cero.

Se descartaron un calentador automático (EventBridge Scheduler) y subir la memoria: para 140 peticiones el efecto es de una sola petición y no justifica una pieza más.

11. Resultados de la medición

11.1 Cómo se midió

Con el script de k6 oficial (1 usuario virtual, 140 iteraciones, pausa de 3 s entre peticiones), cuya única diferencia con el original es la URL del API; el diff de una línea queda guardado como evidencia. El k6 espera hasta el segundo 00 de un minuto para empezar y pide el error en el cuerpo ("error": true) en los bloques 1, 3 y 5 (5, 15 y 15 errores: 35 en total).

La corrida oficial se lanzó desde el panel de control (modo cliente) el 02/11/2026 a las 12:32, hora de Colombia; la primera petición salió a las 12:33:01, al segundo 00 siguiente. La evidencia sale del log y el resumen del k6, del diff del script y de los eventos del router (PETICION, CAMBIO_NIVEL y CORRIDA_NUEVA) exportados de CloudWatch Logs con la ventana de la corrida. El CSV de minutos y el resumen de esa corrida los exportó una versión anterior del panel, que aún no aplicaba la regla de la v3, así que no se usan como evidencia.

11.2 Resultados de la corrida oficial (02/11/2026, AWS)

Medida (resumen de k6)	Valor
Iteraciones completes	140 de 140
Respuestas con error (http_req_failed)	35 de 140 (25 %): son los 5 + 15 + 15 errores que pide el script, sin fallas inesperadas
Latencia promedio, mediana y p95	167 ms, 160 ms y 182 ms

Medida (resumen de k6)	Valor
Latencia de respuestas sin error (promedio y máximo)	168 ms y 356 ms
Latencia máxima de todas las peticiones	356 ms (una respuesta sin error)
Duración promedio por iteración	3,16 s
Latencia del router en el servidor (promedio y máximo)	52,8 ms y 227 ms
Duración de la corrida	7 min 24 s

Cambios de nivel registrados en CloudWatch (corrida oficial del 02/11/2026, 12:32, hora de Colombia):

Petición y hora (Colombia)	Cambio	Motivo
21 · 12:34:04	Nivel 1 → Nivel 2	Degradación (5 errores en el bloque anterior)
61 · 12:36:11	Nivel 2 → Nivel 3	Degradación (15 errores en el bloque anterior)
126 · 12:39:37	Nivel 3 → Nivel 2	Recuperación (racha de 30 sin error)
136 · 12:40:08	Nivel 2 → Nivel 1	Recuperación (racha de 40 sin error)

Por “minuto” del k6 el nivel fue 1, 2, 2, 3, 3, 3 y, en el séptimo, 3, 2 y 1 (5, 10 y 5 peticiones): los cambios ocurrieron en las peticiones 21, 61, 126 y 136, es decir, 1-2-3-2-1, y la corrida terminó en el Nivel 1. El resumen del panel dio la corrida por completa: recorrió los tres niveles, sin saltos, y terminó en el Nivel 1. Los errores por “minuto” del k6 fueron 5, 0, 15, 0, 15, 0 y 0. Los 15 del bloque 5 cayeron mientras el sistema estaba en el Nivel 3, por eso no producen un cambio de nivel; sí rompieron la racha de éxitos, que volvió a empezar en la petición 96 y llegó a 30 en la 126 y a 40 en la 136.

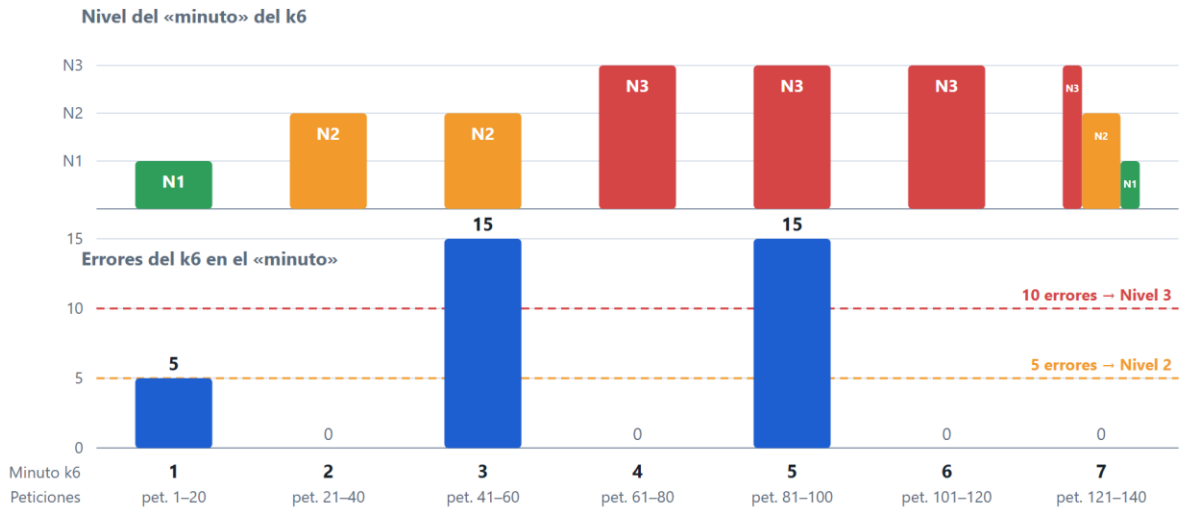


Figura 4. Corrida oficial en AWS (02/11/2026): nivel de servicio de cada "minuto" del k6 (arriba) y errores que inyectó el k6 en ese "minuto" (abajo). Los errores de un "minuto" pueden degradar el siguiente; solo las rachas de éxitos lo recuperan. El "minuto" 7 recorre los tres niveles.

En el servidor, el router tardó en promedio 52,8 ms por petición (mediana de 47,9 ms y percentil 95 de 67,7 ms). El k6 midió 167 ms de promedio porque incluye la red y API Gateway. Los máximos más altos (227 ms en la petición 136 y 167 ms en la petición 1) coinciden con el cambio al Nivel 1 y con el inicio de la corrida; no se midió por separado si fueron arranques en frío. Frente a la versión anterior (unos 37 ms), la v3 cuesta unos 16 ms más por petición, porque hace dos lecturas y dos escrituras en DynamoDB.

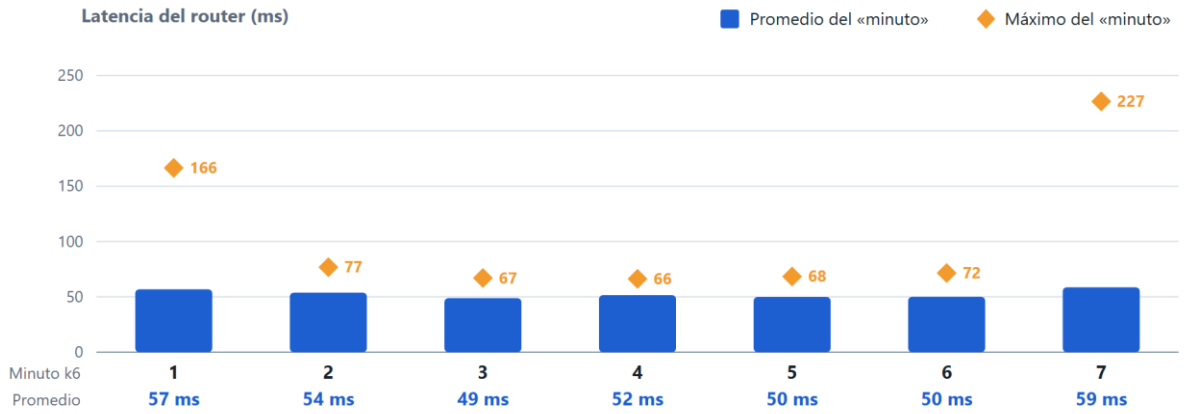


Figura 5. Latencia del router medida en el servidor, por "minuto" del k6: promedio (barras) y máximo (rombos).

12. Disponibilidad: qué pasa si se cae un componente

Si se cae	Mientras está caído	Al Volver
DynamoDB	El router no puede leer ni registrar el estado: responde 503 "Nivel 3: Sistema bajo mantenimiento, intente más tarde" y no cuenta esa petición (evento ESTADO_NO_DISPONIBLE).	Retoma el estado guardado en el ítem V3 (nivel, número de petición y racha); si pasaron más de 15 s sin tráfico, empieza una corrida nueva en el Nivel 1.
Una Lambda de nivel	El router captura la falla (evento FALLA_NIVEL), responde 503 con el mensaje de mantenimiento y cuenta el error.	Los errores dejan de contarse y el sistema se recupera de a un nivel, con las rachas de éxitos.
El router o API Gateway	No hay decisión de nivel y el cliente recibe el error de la puerta de entrada. El diseño no agrega redundancia propia: se apoya en que son servicios gestionados regionales.	Retoma desde el estado guardado en DynamoDB.
CloudWatch Logs o el dashboard	El sistema sigue respondiendo, porque el estado no depende de CloudWatch. Se pierde la visibilidad mientras dure.	Los eventos nuevos vuelven a registrarse; lo ocurrido en el intervalo no se recupera.

La disponibilidad aquí es la resiliencia ante errores de la propia aplicación; no se diseñó redundancia entre regiones.

13. Conclusión

Responder siempre, aunque sea con menos, fue la idea que ordenó el diseño. Lo que aprendimos: un sistema de recuperación necesita memoria; reaccionar solo al último minuto lo hace oscilar.

Qué	En este reto
Priorizamos	La disponibilidad: el sistema nunca deja de responder, aunque sea con menos funciones.
Decidimos	Degradar de golpe y recuperar con calma: un bloque con errores degrada de inmediato, y para subir se exige una racha de 30 o 40 peticiones sin error, de a un nivel.
Demostramos	La secuencia 1 → 2 → 3 → 2 → 1 en AWS con el script oficial: 140 de 140 peticiones respondidas, 35 con el error que pide el script y cambios de nivel en las peticiones 21, 61, 126 y 136.

Anexo A. Glosario

Término	Definición
Nivel 1, 2 y 3	Los tres niveles de servicio: Full (todas las capacidades), Degradado (solo lo esencial) y Mínimo (mensaje de mantenimiento).
Router	Función Lambda que numera cada petición, decide su nivel, invoca el servicio de ese nivel, cuenta el resultado y registra los cambios.
“Minuto” del k6	Bloque de 20 peticiones numeradas que corresponde a un minuto del script del k6. El sistema no usa el reloj: cuenta peticiones, así que la latencia no mueve las ventanas.
Degradación	Reducir las capacidades del sistema para mantener lo esencial cuando suben los errores.
Falla Segura	Responder de la forma más conservadora (Nivel 3) cuando no se puede conocer el estado.
Reintroducción gradual	Recuperar las capacidades de a un nivel, cuando hay una racha de peticiones sin error, y no de golpe.
Histéresis	Usar una regla para bajar de nivel y otra, más exigente, para subir, de modo que el sistema no oscile: aquí se baja de inmediato y se sube solo tras una racha de éxitos.
Racha de éxitos	Peticiones seguidas sin error; un error la pone en cero. Hacen falta 30 para pasar del Nivel 3 al 2 y 40 para pasar del 2 al 1.
API Gateway	Servicio de AWS que expone la puerta de entrada HTTP; en modo proxy entrega la petición completa al router.
Lambda	Servicio de AWS que ejecuta una función sin administrar servidores.
DynamoDB	Base de datos clave-valor de AWS; aquí guarda, en un solo ítem, el número de petición, el nivel, los errores del bloque y la racha de éxitos.
TTL (time to live)	Tiempo tras el cual DynamoDB borra un ítem; para nuestro caso, 24 horas.
CloudWatch Logs y EMF	Servicio de logs de AWS. EMF (Embedded Metric Format) es un bloque dentro del log que CloudWatch convierte en métricas sin llamar a PutMetricData.
Arranque en frío	Tiempo extra de la primera invocación de una Lambda que lleva un rato sin recibir tráfico.
k6 y VU	Herramienta de pruebas de carga. Un VU (usuario virtual) envía peticiones una tras otra, esperando cada respuesta.
Observabilidad	Capacidad de ver lo que hace el sistema a través de sus logs y métricas.
Táctica de arquitectura	Decisión de diseño puntual, dirigida a mejorar un atributo de calidad concreto.